

**Name: JAYESH OM SAHARAVAT**

**Batch:22**

**Sap ID:590023634**

# **Linear Search vs Binary Search**

## Linear Search

```
linear_search.c (part 1 of 2)

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  int linearSearch(int arr[], int n, int key, long *comparisons) {
6      for (int i = 0; i < n; i++) {
7          (*comparisons)++;
8          if (arr[i] == key) {
9              return i;
10         }
11     }
12     return -1;
13 }
14
15 int main() {
16     int n = 1000;
17     int arr[1000];
18
19     srand(42);
20     for (int i = 0; i < n; i++) {
21         arr[i] = rand() % 10000;
22     }
23
24     printf("Generated %d random numbers.\n", n);
25     printf("First 10 elements: ");
26     for (int i = 0; i < 10; i++) {
27         printf("%d ", arr[i]);
28     }
29     printf("\n\n");
30
31     int keysToFind[5];
32     for (int i = 0; i < 5; i++) {
33         keysToFind[i] = arr[(i * n) / 5];
34     }
35     int missingKey = -999;
36
37     double totalTime = 0.0;
38     long totalComparisons = 0;
39
40     for (int i = 0; i < 5; i++) {
41         long comparisons = 0;
42         clock_t start = clock();
```

```

linear_search.c (part 2 of 2)

43     int index = linearSearch(arr, n, keysToFind[i], &comparisons);
44     clock_t end = clock();
45     double timeTaken = ((double)(end - start)) / CLOCKS_PER_SEC * 1000.0;
46
47     printf("Search %d: key = %d, found at index = %d, comparisons = %ld, time = %.6f ms\n",
48           i + 1, keysToFind[i], index, comparisons, timeTaken);
49
50     totalTime += timeTaken;
51     totalComparisons += comparisons;
52 }
53
54 long comparisons = 0;
55 clock_t start = clock();
56 int index = linearSearch(arr, n, missingKey, &comparisons);
57 clock_t end = clock();
58 double timeTaken = ((double)(end - start)) / CLOCKS_PER_SEC * 1000.0;
59
60 printf("Search 6 (missing key): key = %d, found at index = %d, comparisons = %ld, time = %.6f ms\n",
61       missingKey, index, comparisons, timeTaken);
62
63 totalTime += timeTaken;
64 totalComparisons += comparisons;
65
66 printf("\nAverage comparisons per search: %.2f\n", (double)totalComparisons / 6.0);
67 printf("Average time per search: %.6f ms\n", totalTime / 6.0);
68
69 int repeats = 100000;
70 long dummyComparisons = 0;
71 clock_t benchStart = clock();
72 for (int r = 0; r < repeats; r++) {
73     linearSearch(arr, n, missingKey, &dummyComparisons);
74 }
75 clock_t benchEnd = clock();
76 double benchTime = ((double)(benchEnd - benchStart)) / CLOCKS_PER_SEC * 1000.0;
77 printf("\nBenchmark: %d worst-case searches took %.3f ms total, %.6f ms per search\n",
78       repeats, benchTime, benchTime / repeats);
79
80 return 0;
81 }

```

## Linear Search

Generated 1000 random numbers.

First 10 elements: 6166 2740 8881 3241 1012 6758 9021 4940 535 3743

Search 1: key = 6166, found at index = 0, comparisons = 1, time = 0.001000 ms

Search 2: key = 9044, found at index = 200, comparisons = 201, time = 0.001000 ms

Search 3: key = 2906, found at index = 400, comparisons = 401, time = 0.001000 ms

Search 4: key = 6337, found at index = 600, comparisons = 601, time = 0.002000 ms

Search 5: key = 157, found at index = 800, comparisons = 801, time = 0.002000 ms

Search 6 (missing key): key = -999, found at index = -1, comparisons = 1000, time = 0.002000 ms

Average comparisons per search: 500.83

Average time per search: 0.001500 ms

Benchmark: 100000 worst-case searches took 144.983 ms total, 0.001450 ms per search

## Binary Search

```
binary_search.c (part 1 of 3)

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  int compareInts(const void *a, const void *b) {
6      return (*(int *)a - *(int *)b);
7  }
8
9  int binarySearch(int arr[], int n, int key, long *comparisons) {
10     int low = 0, high = n - 1;
11     while (low <= high) {
12         (*comparisons)++;
13         int mid = low + (high - low) / 2;
14         if (arr[mid] == key) {
15             return mid;
16         } else if (arr[mid] < key) {
17             low = mid + 1;
18         } else {
19             high = mid - 1;
20         }
21     }
22     return -1;
23 }
24
25 int main() {
26     int n = 1000;
27     int arr[1000];
28
29     srand(42);
30     for (int i = 0; i < n; i++) {
31         arr[i] = rand() % 10000;
32     }
33
34     qsort(arr, n, sizeof(int), compareInts);
35
36     printf("Generated and sorted %d random numbers.\n", n);
37     printf("First 10 elements after sorting: ");
38     for (int i = 0; i < 10; i++) {
39         printf("%d ", arr[i]);
40     }
41     printf("\n\n");
```



# binary\_search.c (part 2 of 3)

```
43     int keysToFind[5];
44     for (int i = 0; i < 5; i++) {
45         keysToFind[i] = arr[(i * n) / 5];
46     }
47     int missingKey = -999;
48
49     double totalTime = 0.0;
50     long totalComparisons = 0;
51
52     for (int i = 0; i < 5; i++) {
53         long comparisons = 0;
54         clock_t start = clock();
55         int index = binarySearch(arr, n, keysToFind[i], &comparisons);
56         clock_t end = clock();
57         double timeTaken = ((double)(end - start)) / CLOCKS_PER_SEC * 1000.0;
58
59         printf("Search %d: key = %d, found at index = %d, comparisons = %ld, time = %.6f ms\n",
60             i + 1, keysToFind[i], index, comparisons, timeTaken);
61
62         totalTime += timeTaken;
63         totalComparisons += comparisons;
64     }
65
66     long comparisons = 0;
67     clock_t start = clock();
68     int index = binarySearch(arr, n, missingKey, &comparisons);
69     clock_t end = clock();
70     double timeTaken = ((double)(end - start)) / CLOCKS_PER_SEC * 1000.0;
71
72     printf("Search 6 (missing key): key = %d, found at index = %d, comparisons = %ld, time = %.6f ms\n",
73         missingKey, index, comparisons, timeTaken);
74
75     totalTime += timeTaken;
76     totalComparisons += comparisons;
77
78     printf("\nAverage comparisons per search: %.2f\n", (double)totalComparisons / 6.0);
79     printf("Average time per search: %.6f ms\n", totalTime / 6.0);
80
81     int repeats = 100000;
82     long dummyComparisons = 0;
83     clock_t benchStart = clock();
84     for (int r = 0; r < repeats; r++) {
```



# binary\_search.c (part 3 of 3)

```
85         binarySearch(arr, n, missingKey, &dummyComparisons);
86     }
87     clock_t benchEnd = clock();
88     double benchTime = ((double)(benchEnd - benchStart)) / CLOCKS_PER_SEC * 1000.0;
89     printf("\nBenchmark: %d worst-case searches took %.3f ms total, %.6f ms per search\n",
90         repeats, benchTime, benchTime / repeats);
91
92     return 0;
93 }
```

Generated and sorted 1000 random numbers.

First 10 elements after sorting: 18 26 46 51 58 61 81 86 90 111

Search 1: key = 18, found at index = 0, comparisons = 9, time = 0.002000 ms

Search 2: key = 1951, found at index = 200, comparisons = 10, time = 0.000000 ms

Search 3: key = 3923, found at index = 400, comparisons = 10, time = 0.001000 ms

Search 4: key = 6013, found at index = 600, comparisons = 7, time = 0.000000 ms

Search 5: key = 8072, found at index = 800, comparisons = 10, time = 0.000000 ms

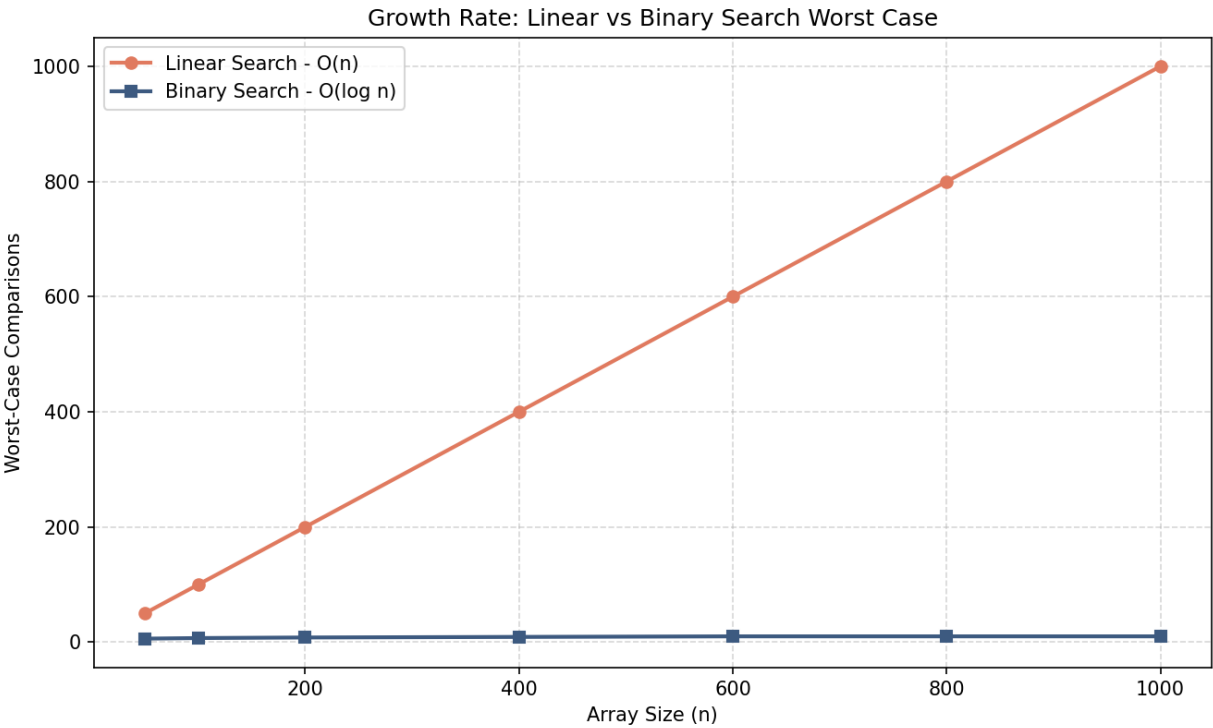
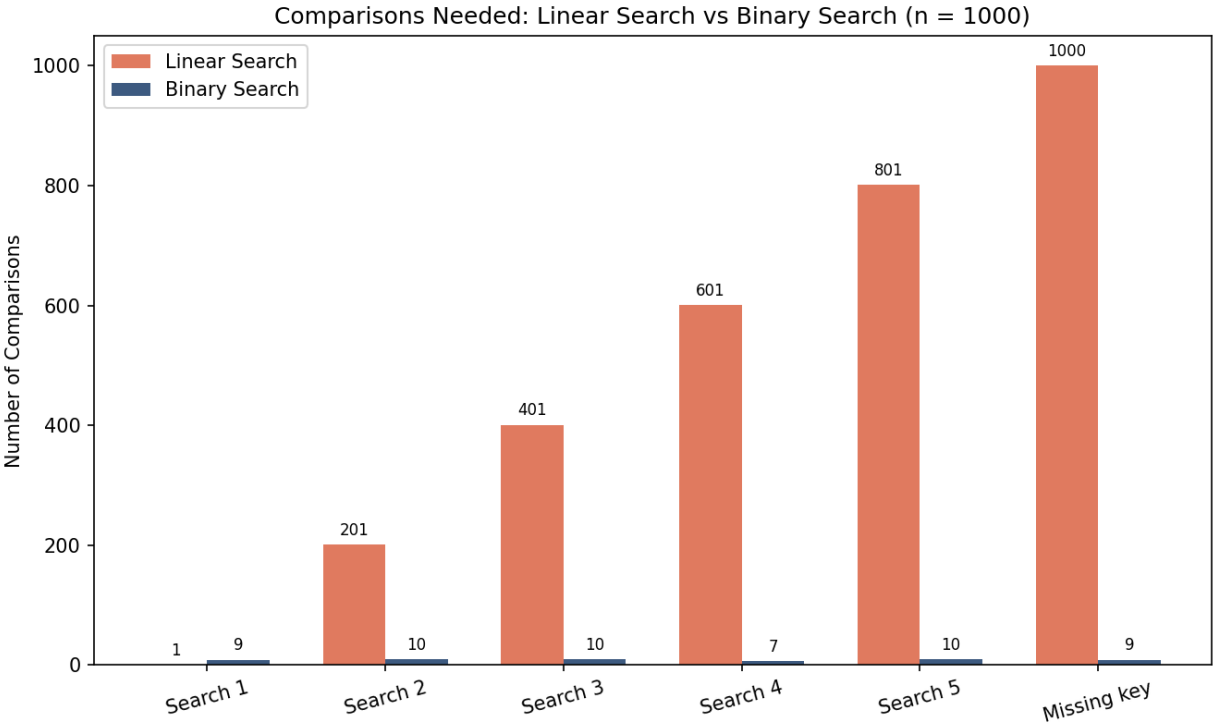
Search 6 (missing key): key = -999, found at index = -1, comparisons = 9, time = 0.001000 ms

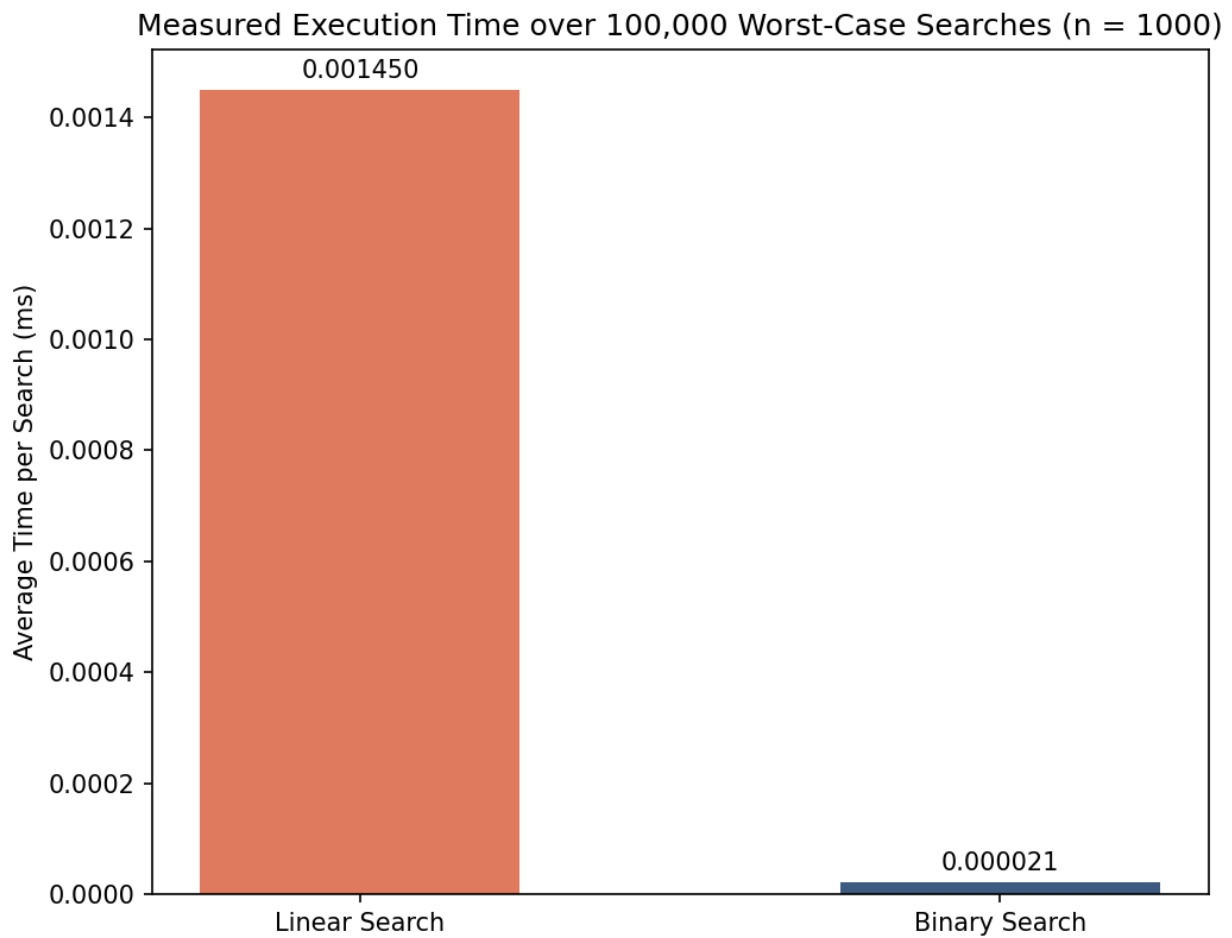
Average comparisons per search: 9.17

Average time per search: 0.000667 ms

Benchmark: 100000 worst-case searches took 2.116 ms total, 0.000021 ms per search

Graphical Comparison





## Understanding

Binary search is superior to linear search for large datasets because its  $O(\log n)$  time complexity dramatically reduces lookups compared to linear search's  $O(n)$  linear scan. However, binary search requires  $O(n \log n)$  upfront time (and potentially additional space) to sort the data first, making linear search the more efficient, zero-overhead  $O(1)$  choice for small lists or one-off searches.

